

CSA1006 – SOFTWARE ENGINEERING

Name: Pravin Raajhan K

Reg no.: 192521066

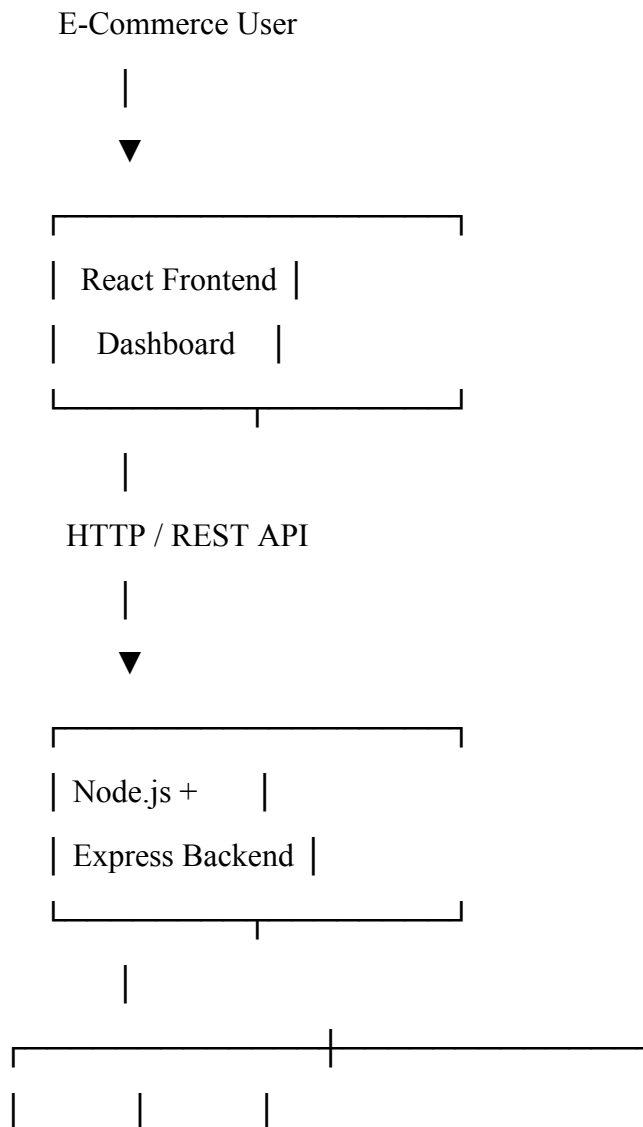
UNIT – 3 Assessment

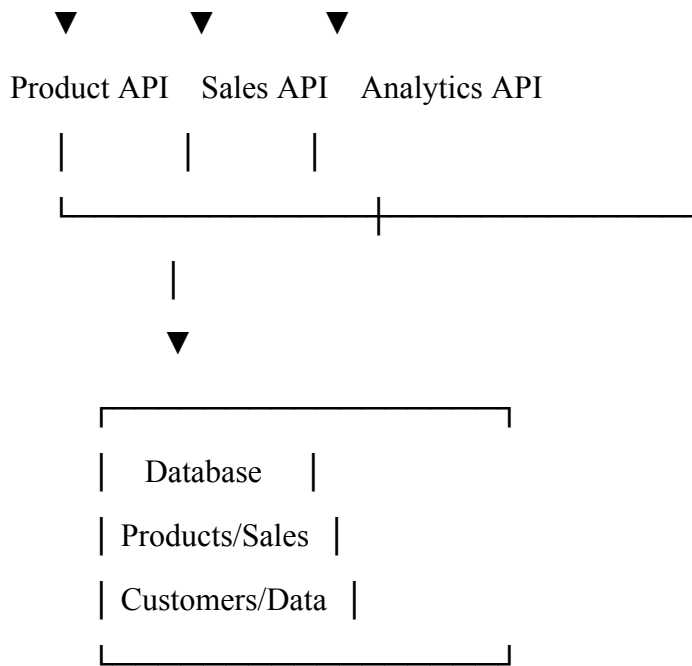
Problem 1: Full-Stack Web App Development for an E-Commerce Dashboard

1. System Architecture

The proposed e-commerce analytics dashboard uses **React** for the frontend, **Node.js with Express** for the backend API, and a database for storing application data.

Architecture Flow





Component Description

React Frontend

- Provides the user interface for the analytics dashboard.
- Displays sales figures, product performance, customer information, and interactive charts.
- Sends requests to the backend through REST APIs.

Node.js + Express Backend

- Acts as the application and API layer.
- Receives requests from the React frontend.
- Processes business logic and communicates with the database.
- Returns the required data in a structured format such as JSON.

Database

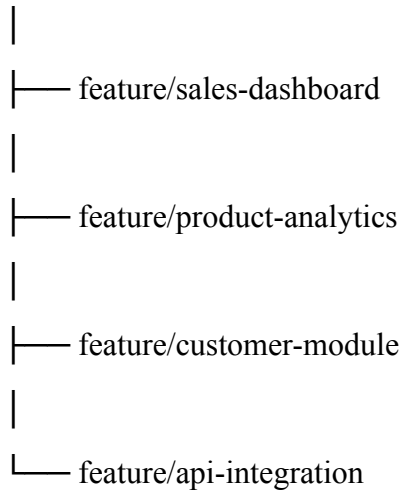
- Stores product, customer, sales, order, and analytics-related information.
- The backend retrieves or updates this information based on frontend requests.

2. Git Branching Strategy

For this project, **Feature Branching** is recommended because multiple developers will work on different dashboard components.

Branch Structure

main



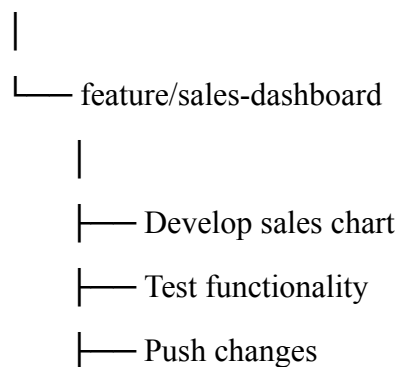
Workflow

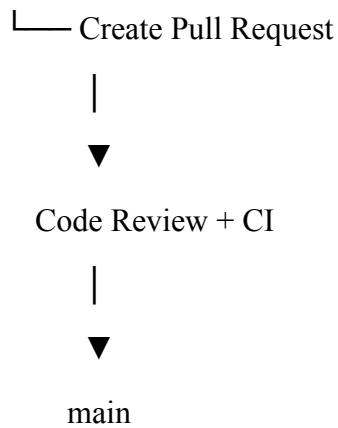
1. The main branch contains stable and reviewed code.
2. A developer creates a separate feature branch for each new functionality.
3. The developer implements and tests the feature locally.
4. The feature branch is pushed to the remote repository.
5. A **Pull Request (PR)** is created to merge the feature into main.
6. Another developer reviews the code.
7. Automated CI checks are performed.
8. If the review and CI checks are successful, the PR is merged.
9. The feature branch can then be deleted.

Example

If a developer is working on sales charts:

main





This approach prevents developers from directly modifying stable code and makes collaboration safer.

3. API Data Flow

The API acts as the communication layer between the React frontend and the backend/database.

Data Flow

User



React Dashboard



| HTTP Request



Express REST API



Business Logic



Database



| Retrieved Data



Express API



| JSON Response



React Dashboard



Charts / Tables / Reports

Example: Sales Data Request

Suppose the manager wants to view the **monthly sales report**.

1. The manager opens the sales dashboard.
2. React sends a request to the backend.
3. Express receives the API request.
4. The backend processes the request and queries the database.
5. The database returns the required sales information.
6. Express converts the result into a JSON response.
7. React receives the response.
8. The dashboard displays the information using charts or tables.

Example API Endpoint

GET /api/sales/monthly

Example response:

```
{  
  "month": "August",  
  "totalSales": 125000,  
  "orders": 1840  
}
```

The React application can use this information to generate an interactive sales chart.

4. CI Flow

Continuous Integration helps ensure that code submitted by developers does not break the existing application.

Proposed CI Flow

Developer

|



Feature Branch

|



Push Code

|



Create Pull Request

|



CI Pipeline

|

|— Install Dependencies

|

|— Linting

|

|— Unit Tests

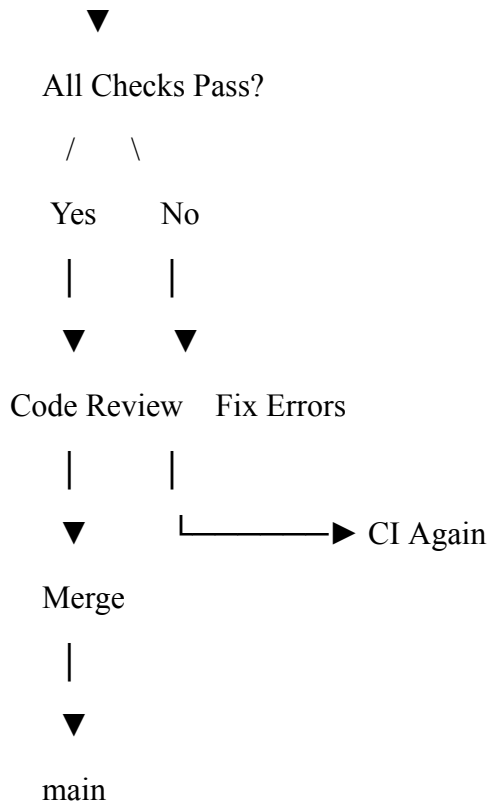
|

|— Build Application

|

|— Integration Tests

|



Benefits of CI

- Detects errors early.
- Prevents broken code from being merged.
- Automatically performs testing.
- Improves code quality.
- Makes collaboration between developers easier.
- Reduces the chance of deployment failures.

5. Example Pull Request Review Checklist

Before merging a pull request, the reviewer can use the following checklist:

Check	Requirement
Code Quality	Is the code clean, readable, and properly structured?
Functionality	Does the feature work according to the requirements?
Testing	Are appropriate unit/integration tests included?
UI	Does the React interface work correctly on the dashboard?

Check	Requirement
API	Are API requests and responses handled correctly?
Error Handling	Are failures and invalid requests handled properly?
Security	Are sensitive data and credentials protected?
Performance	Does the feature avoid unnecessary API calls or processing?
CI Status	Have all automated CI checks passed?
Documentation	Are important changes documented clearly?

Pull Request Approval Rule

A pull request should be merged only when:

Code Review Approved + CI Checks Passed + Tests Passed

This ensures that only reviewed and tested code enters the stable main branch.

6. Overall Development Workflow

The complete workflow for the e-commerce dashboard can be summarized as:

Requirement

|



Create Feature Branch

|



Develop React / Node.js Feature

|



Test Locally

|



Push to Git Repository

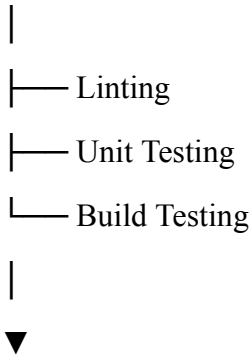
|



Create Pull Request



CI Checks



Code Review



Merge into Main



Updated E-Commerce Dashboard

Conclusion

The proposed full-stack architecture combines **React, Node.js/Express, REST APIs, and a database** to create an efficient e-commerce analytics dashboard. React provides an interactive interface for displaying charts and reports, while Node.js and Express handle API requests and business logic. A feature-based Git workflow allows developers to work independently without affecting stable code. Continuous Integration automatically checks the submitted code through linting, testing, and build verification before merging. Finally, mandatory pull-request reviews help maintain code quality, security, and reliability throughout the development process.